VERILOG-AMS VERIFICATION METHODOLOGY OF THE CONTROL BITS FOR A MIXED-SIGNAL TRANSCEIVER

A Project

Presented to the faculty of the Department of Electrical and Electronic Engineering

California State University, Sacramento

Submitted in partial satisfaction of
the requirements for the degree of

MASTER OF SCIENCE

in

Electrical and Electronic Engineering

by

Ashwin Kantharaj

FALL
2017

VERILOG-AMS VERIFICATION METHODOLOGY OF THE CONTROL BITS

FOR MIXED-SIGNAL TRANSCEIVER

A Project

by

Ashwin Kantharaj

Approved by:

_____, Committee Chair
Perry L. Heedley, Ph.D.

_____, Second Reader
Todd Honan

_____
Date

Student: Ashwin Kantharaj

I certify that this student has met the requirements for format contained in the University format manual, and that this project is suitable for shelving in the Library and credit is to be awarded for the project.

_____, Graduate Coordinator _____
Preetham B. Kumar, Ph. D. Date

Department of Electrical and Electronic Engineering

Abstract

of

VERILOG-AMS VERIFICATION METHODOLOGY OF THE CONTROL BITS

FOR A MIXED-SIGNAL TRANSCEIVER

by

Ashwin Kantharaj

Connectivity verification on a complex and large-scale chip can be a daunting and one of the crucial tasks to ensure the silicon to work on the first attempt of fabrication. Mixed-signal connectivity originates at a digital control bit and terminates at an analog node. Connectivity is verified on the entire hierarchy and not just at the boundary of analog and digital domains. The Verilog-AMS framework adopted for this methodology provides the ease of using a SystemVerilog or a Universal Verification Methodology (UVM) based testbench that is widely used for verification of the digital design. This methodology supports connectivity verification in an existing testbench environment without the need of analog-digital co-simulation setup.

_____, Committee Chair
Perry L. Heedley, Ph. D.

_____
Date

ACKNOWLEDGEMENTS

I would like to thank Dr. Perry L. Heedley for taking up the role of the committee chair for this project. His feedback and guidance were essential for me to complete this project successfully.

I would like to thank Todd L. Honan, DV Manager, Analog Devices, Inc. (ADI), for allowing me to work on this project at ADI and incorporating this project as part of my internship. Without ADI's CAD infrastructure and Todd's support, this project would not have been this successful.

I would also like to thank Mixed-Signal Expert Dushyant Juneja and Digital DV Expert Jainik Kathiara for their continuous support and technical help throughout this project.

Lastly, special thanks to my lovely wife Mithila for her patience and support she gave me the whole time.

TABLE OF CONTENTS

Page

Acknowledgments vi

List of Figures ix

Chapter

1. INTRODUCTION…….…….. 1

    1.1 Background 1

    1.2 Statement of Collaboration..... 2

    1.3 Proposal 3

2. VERILOG-AMS ENDPOINT ASSERTIONS METHODOLOGY 4

    2.1 Methodology Requirements 4

    2.2 Methodology flow 4

    2.3 Mixed Signal Design 5

    2.4 Mixed Signal Hierarchy 7

    2.5 Design preparation 8

    2.6 Endpoint Checker 9

    2.7 Counter output in Digital and Analog Domains..... 10

3. IMPLEMENTATION ON TRANSCEIVER PROJECT 12

    3.1 Transceiver Register Implementation 12

    3.2 Netlist… 13

    3.3 Analog_top 14

    3.4 West block..... 14

    3.5 Cadence runams netlist flow 15

    3.6 Endpoint Assertions 16

3.7 Cadence $cadence_get_analog_value() or $cgav function 18

3.8 Assertion binding 18

3.9 Compile and Simulation 19

3.10 Analog Control File amscf.scs 19

3.11 Bit Bash Test 20

4. RESULTS AND OBSERVATIONS 22

4.1 Compilation and Elaboration 22

4.2 Simulation Results 22

5. CONCLUSIONS AND FUTURE WORK 23

Appendix A. ENDPOINT ASSERTION MACRO 24

Appendix B. XCELIUM XRUN –F ARGUMENT FILE 25

Appendix C. ANALOG SOLVER CONTROL FILE AMSCF.SCS 27

References 28

LIST OF FIGURES

Figures Page

1. Typical Mixed signal Transceiver system 2

2. Schematic of entity chip_top 5

3. Schematic of entity sim_chip_top 6

4. Schematic of the entity analog_top 6

5. Mixed-Signal Hierarchy 7

6. Hierarchy Editor Tool showing config view of chip_top 8

7. SVA checker for the digital outputs that control NMOS switches 9

8. Simulation plot of Counter output in both Digital and Analog Domains 11

9. Register access to analog sub-map through external SPI master.....12

10. Analog sub-map Register access with multiple masters 12

11. Analog_top netlist with the instantiation of West block..... 14

12. Runams netlister command-line options..... 15

13. West block netlist extract 16

14. Bind statement for analog block named aux_pll 19

Chapter 1

INTRODUCTION

1.1 Background

The cost of fabrication process has increased exponentially with the advancement in the latest sub-micron process nodes [1]. Higher costs have driven the demand for a higher yield per silicon wafer and the first silicon to be fully functional and close to the production version. Such confidence in the design is achieved by efficient and exhaustive verification techniques. With advancements in Computer Aided Design (CAD) tools, companies have been able to develop large designs with multi-domain blocks to fit on a single die. Analog-Digital co-simulation environments, using a digital simulator and an analog solver concurrently, have enabled engineers to implement more efficient verification framework for complex designs. These advancements come with myriad of drawbacks and challenges [2].

A complex mixed-signal Radio Frequency (RF) chip consists of numerous digital, analog and RF blocks distributed across the chip. A generic RF chip architecture is as shown in Figure 1. A typical control bit resides in one of the digital processing blocks or memory. The digital definition of the control bit starts at a digital register and ends as an output port on the hierarchically topmost digital block. The analog definition of the bit begins at that boundary, spans across the analog circuit hierarchy, and terminates at the gate of a transistor.

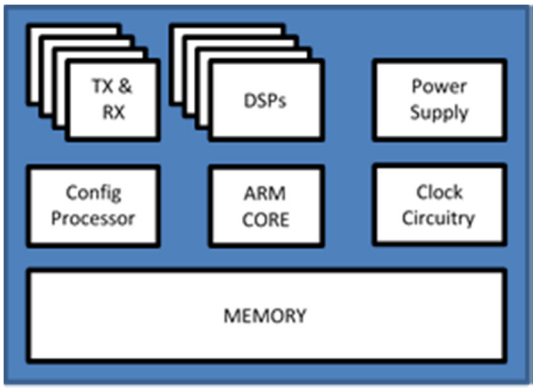

Figure 1. Typical Mixed signal Transceiver system

A typical analog-digital co-simulation setup can verify a control bit in two stages. Connectivity from the control bit origin to the output port of the digital block is verified in a digital environment. The digital hierarchy is simulated using a digital simulator or a Formal Property verifier, and the connectivity is checked off as part of toggle coverage. Analog designers set up analog simulations in a Graphical User Interface (GUI) tool like Cadence Virtuoso. An analog solver, like SPICE or Spectre, simulate the analog design and the results are obtained in the form of waveforms. Analog solvers are continuous-time engines that solve complex mathematical operations and take extremely long to simulate even smaller time units like a few microseconds. Such a co-simulation setup is not an efficient solution for connectivity verification, which requires read and writes operations to analog control bits address space.

1.2 Statement of collaboration

ADI employs a SPICE-based homegrown analog solver to simulate analog designs. Many projects at ADI use this simulator for analog simulations and co-simulation of mixed-signal chips at the system level. However, for an exhaustive verification such as connectivity of control bits, a co-simulation setup has proven to be an inefficient method. Such a setup poses various challenges

due to homegrown nature of simulator that can cause conflicts with the digital simulator that runs concurrently.

This project was initiated at ADI to explore a more efficient environment and to establish a methodology for connectivity verification and overcome shortfalls of the existing setup. The results of this project are analyzed in contrast with the existing co-simulation setup to evaluate that the methodology captures the objective without leaving any loopholes in verification. The project is carried out on an ongoing complex mixed-signal transceiver chip with multiple transmit, receive, and observation channels with multiple processors and signal processing units.

1.3 Proposal

The outcome of this project is a methodology that defines the procedure for verifying the end-point connections for analog control bits. The methodology is based on Cadence's Analog and mixed-signal simulation (AMS) flow. The AMS flow employs a Verilog-AMS based netlist for analog design as opposed to SPICE circuit definitions. The methodology aims to exploit the Verilog-AMS language's compatibility with Verilog and SystemVerilog languages and Cadence's AMS flow, a single tool simulation framework. The methodology is mostly kept to be a command based batch-mode procedure to accommodate the ability to support existing testbench environments.

Chapter 2

VERILOG-AMS ENDPOINT ASSERTIONS METHODOLOGY

2.1 Methodology requirements

This methodology applies to designs that have analog blocks developed using Virtuoso or similar schematic tool. The design can be either pure analog or a mixed signal hierarchical design. The testbench for this project is a SystemVerilog Universal Verification Methodology (UVM) based setup with the digital design modeled at Register Transfer Level (RTL). As mentioned in the previous chapter, this methodology uses AMS flow for netlisting of analog circuitry and simulation. The latest version of Cadence Incisive or Xcelium tool suite is required. Any other setup requirements such as compilers or assemblers for any processors on the chip are out of the scope of this project and assumed to be installed and configured.

2.2 Methodology Flow

The methodology flow defines the procedure to obtain a Verilog-AMS netlist for the entire analog hierarchy and integration with the digital testbench setup for simulation. The methodology also defines the assertion framework to ensure the endpoint connectivity is thoroughly verified. The automated assertion generation is embedded into this process. The final step of the methodology is inspecting assertion coverage for the Endpoints and signoff.

As a proof of concept for this methodology, a smaller scale design is used in the following sections on which the methodology is applied. The large-scale implementation and observations made are explained in more detail in the following chapters.

2.3 Mixed Signal Design

A mixed-signal design is a combination of analog and digital circuitry. Various kinds of such design and different proportions of analog and digital blocks, based on system requirements, are explained by Mladen Nizic in chapter 1 of Mixed-Signal Methodology Guide [3]. Unlike digital counterpart, analog signals can change in almost infinitely small steps in terms of time and amplitude. Analog functionality is usually described with nonlinear equations. Traditionally, analog verification is performed using SPICE simulations, capable of solving nonlinear equations. Whereas digital circuits are simulated by solving Boolean equations, which are much faster [3].

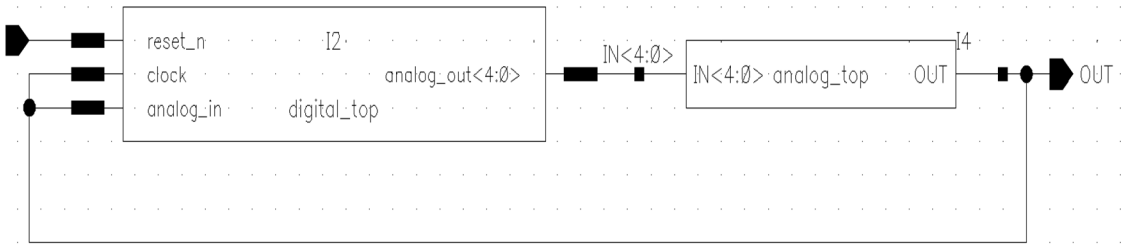

Figure 2. Schematic of entity chip_top

Figure 2 shows the schematic of the top level entity of the system named chip_top. The chip_top contains two lower level blocks called digital_top and analog_top with connections running from both analog to digital and digital to analog domains. The transceiver schematic discussed later in the project has thousands of bits exchanged between analog and digital.

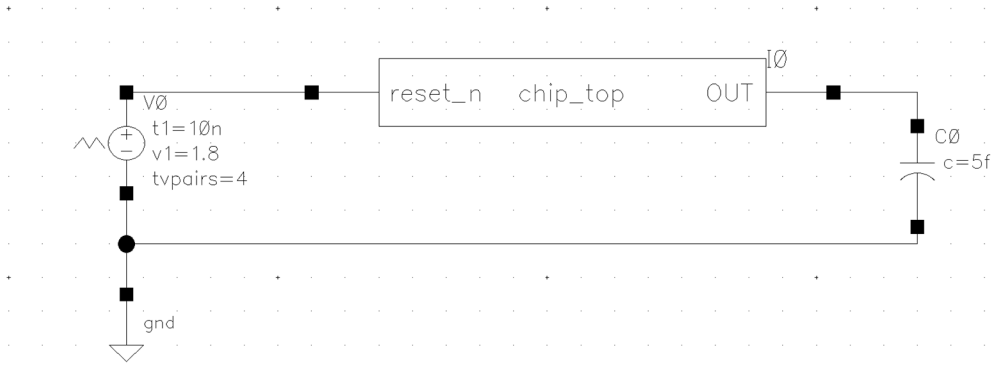

Figure 3. Schematic of entity sim_chip_top

The top-level simulation entity is as shown in Figure 3. It has an instance of the design under test chip_top and other components to support simulation.

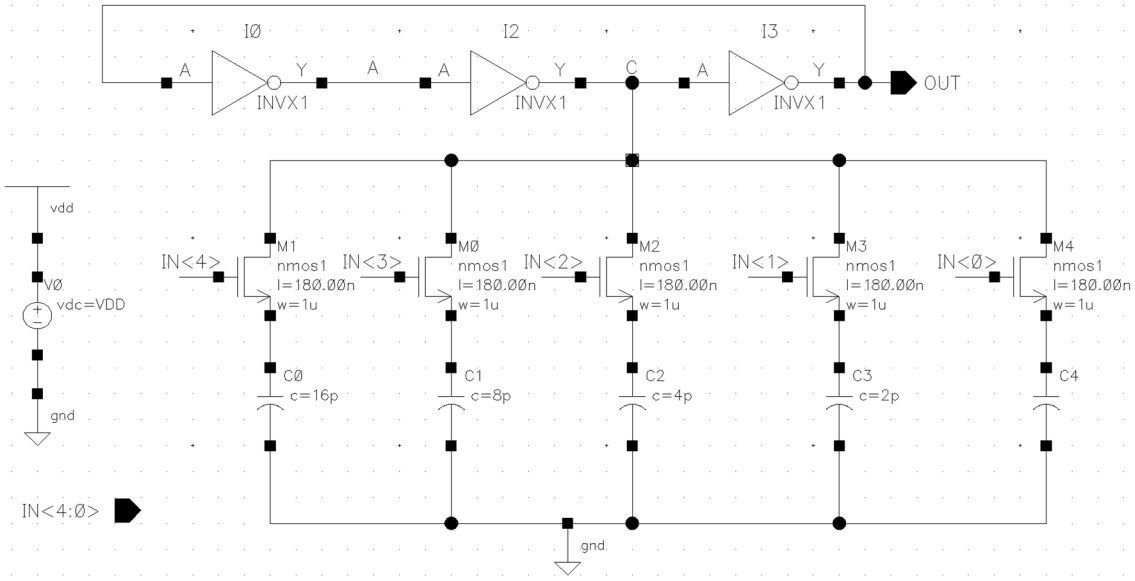

Figure 4. Schematic of the entity analog_top

As shown in Figure 4, analog_top is an oscillator circuit with a digitally variable capacitance at node C. N-channel Metal Oxide Semiconductor Field Effect Transistor (NMOS)

switches turn on or off a capacitor array with capacitors with capacitances of powers of two. A digital counter periodically varies the capacitance on node C by controlling the NMOS switches in an incremental order. Any digital logic could be used to control these switches. The counter is an arbitrary choice for this example. The digital block is left to be just a symbol. Appropriate definition for the digital_top is filled in using RTL definition or a Gate level definition. A Hardware Description Language (HDL), like Verilog or VHDL, is used to describe the digital_top block.

2.4 Mixed-Signal Hierarchy

Figure 5 shows the hierarchy of the entire environment. Sim_chip_top is the topmost hierarchical block with instances of the design block chip_top, simulation components C0 and V0. Further down the hierarchy, chip_top consists of instances of analog_top and digital_top with connections as shown in Figure 2.

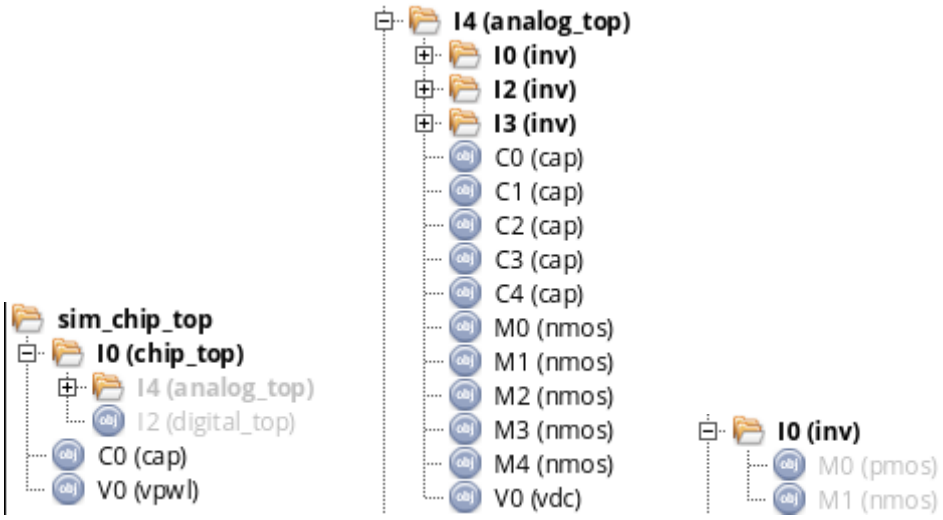

Figure 5. Mixed-Signal Hierarchy

The analog_top has instances of primitive components NMOS and Capacitors, along with instances of another higher-level component INVX1. INVX1 is functionally an inverter that has one instance of primitives NMOS and PMOS each. Adding to the complexity, the inverter INVX1 has multiple definitions. The inverter has definitions in a schematic hierarchy as shown in Figure 5, a Verilog RTL behavioral description, and a Verilog-AMS analog behavioral description.

2.5 Design preparation

The first step of the methodology is preparing the design for netlisting and simulation. Cadence Virtuoso offers a mechanism for defining the hierarchy called config views. Hierarchy Editor Tool is used to create config views to create the hierarchy definitions by selecting appropriate definitions or views for each instance of every component in the design. Figure 6 shows the config view of the top-level design block chip_top.

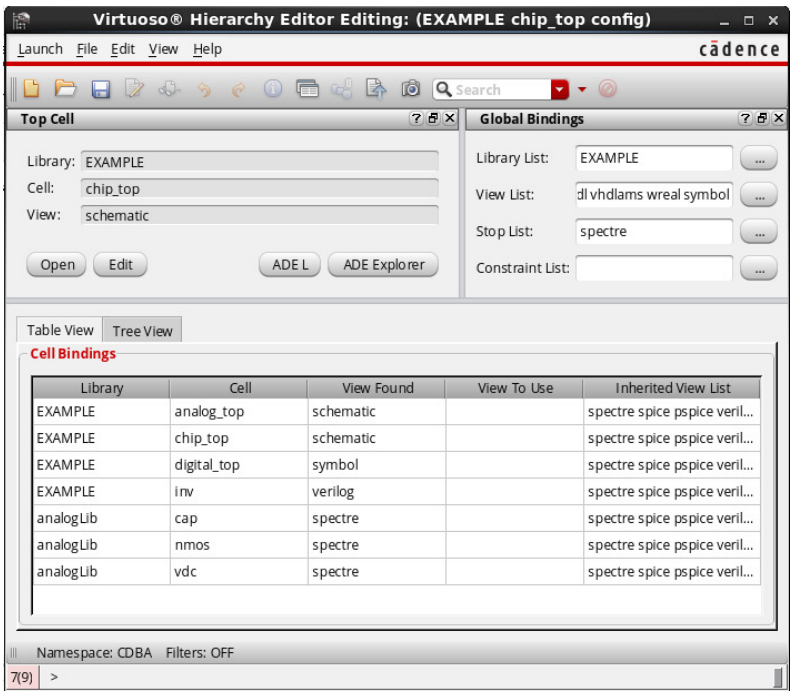

Figure 6. Hierarchy Editor Tool showing config view of chip_top

The Virtuoso Tool offers a template for creation of config view that supports AMS simulation flow. The View List and Stop List are allowed to be modified to suit the design better. The views or types of description are selected in this window for each cell. Any cell that need not be part of the netlist is bound to open. The netlisting tool skips the cells that are bound to open.

For a setup that relies entirely on Virtuoso tool for simulation would require a config view for the top-level entity sim_chip_top. The lower level hierarchical blocks that have their own config views can have the config as the view to be used by the netlister. To keep the methodology compatible with existing designs, batch-mode flow is adopted. Hence, the topmost hierarchical block fed to the netlister is chip_top.

2.6 Endpoint checker

An entirely separate module is created with checks that verify the connectivity of the control bits. The module is compiled with the top-level block sim_chip_top and the signals in the sim_chip_top block are accessible by the assertion module using hierarchical name lookup. SystemVerilog Assertions (SVA) are used to monitor the connectivity continuously. Concurrent assertions as explained in [4] assert the specified behavior on a signal or a collection of signals. The simulator throws an error whenever the assertion fails. Assertions are defined for every endpoint connection. Every check is defined as a property that is monitored continuously.

```
property dig analog;
        real value old, value new;
        @(posedge sim chip top.I0.I2.clock) (1, value old = $cgav("sim chip top.I0.I4.IN")) |=>
        (1, value new = $cgav("sim chip top.I0.I4.IN")) ##0 (value new == value old) ;
endproperty

   epa dig analog : assert property (dig analog);
```

Figure 7. SVA checker for the digital outputs that control NMOS switches

The assertion shown in Figure 7 is hardcoded to monitor the digital output of the digital_top block that travels through the hierarchy and reaches the gate nodes in the analog_top block. Since the digital block is a counter in this example, the assertion makes sure that the digital output changes every clock cycle and no two consecutive values are same.

A more complex assertion framework, triggered by read and write operations to the control bits, is explained in chapter 3. A massive system with hundreds or even thousands of control bits requires an automated mechanism for generation of assertions.

2.7 Counter output in Digital and Analog Domains

Figure 8 shows the counter signals on both digital and analog domains. The signal bus analog_out is the 5-bit output of digital_top that increments, by value 1, every clock cycle. The signals IN[0] through IN[4] are the same signals that terminate in the analog domain, at the gate terminals of NMOS switches. The analog oscillations on the signal OUT of analog_top serve as the clock input to the digital_top block. The signal is plotted in the digital domain and can be seen incrementing every clock cycle.

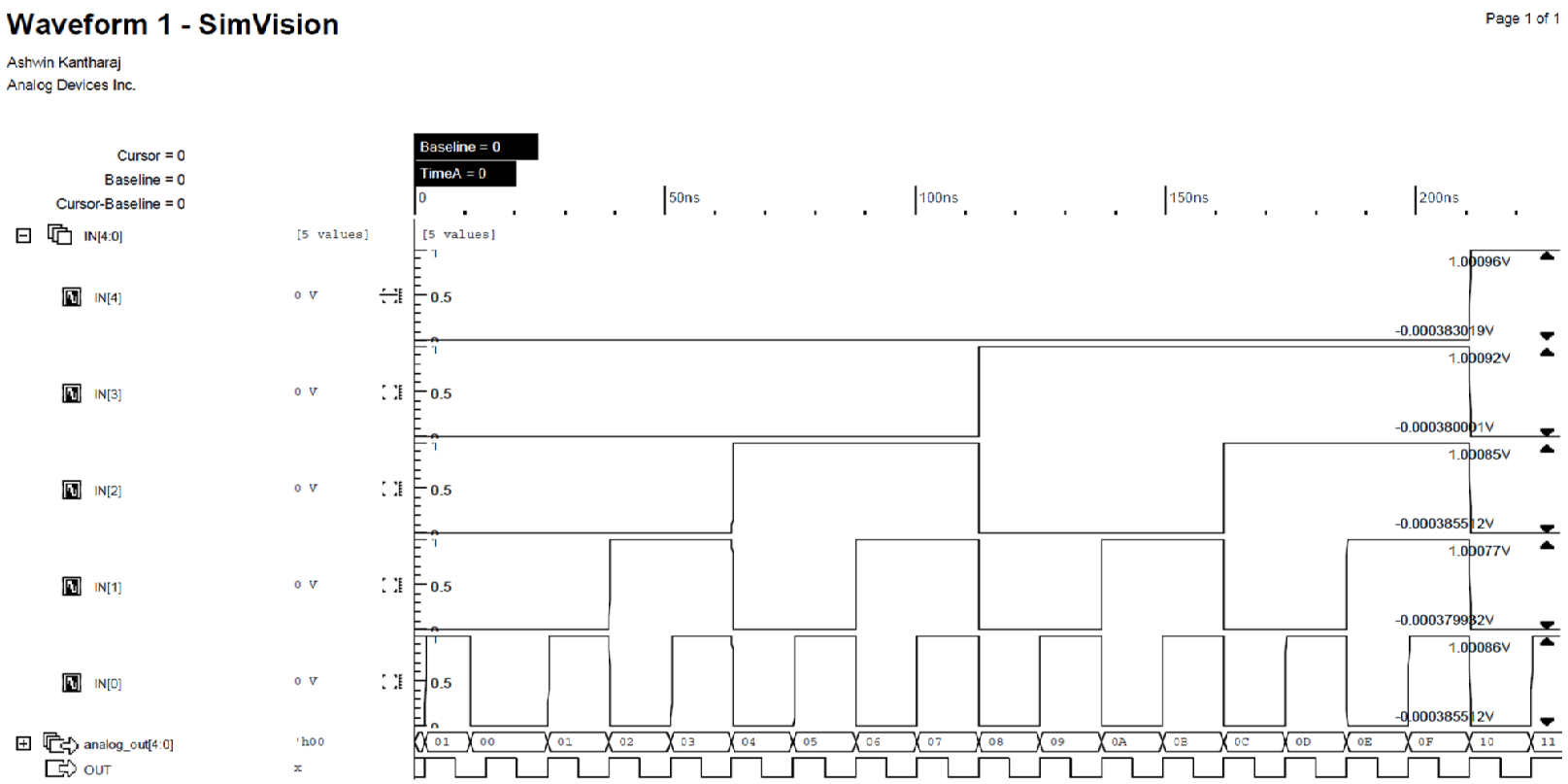

Figure 8. Simulation plot of Counter output in both Digital and Analog Domains

Chapter 3

IMPLEMENTATION ON TRANSCEIVER PROJECT

3.1 Transceiver Register implementation

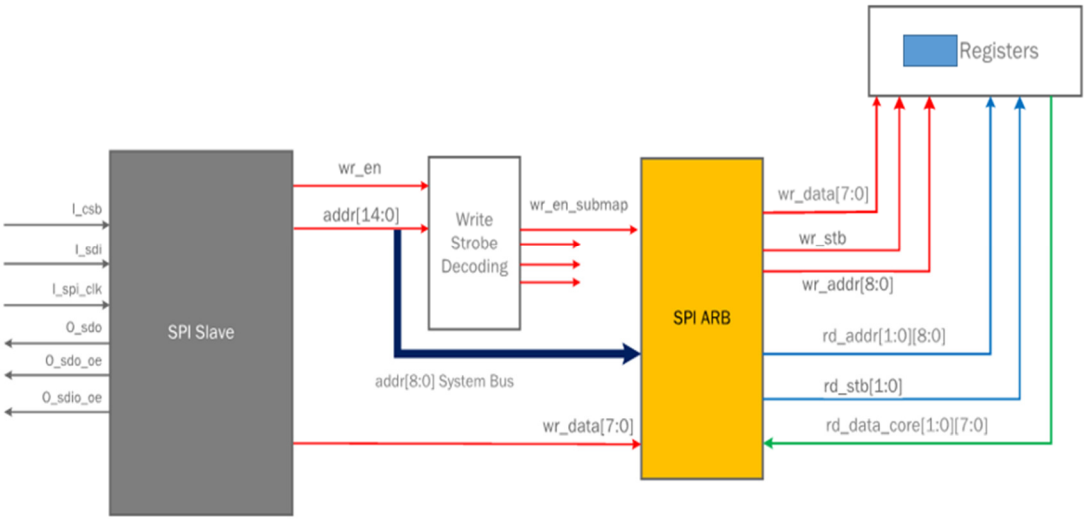

Figure 9. Register access to analog sub-map through external SPI master

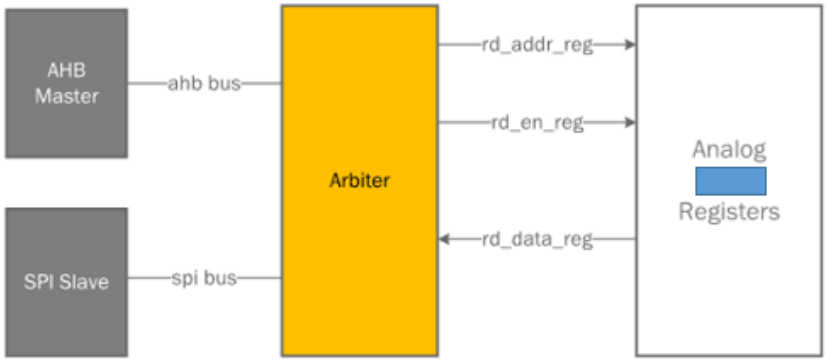

Figure 10. Analog sub-map Register access with multiple masters

The Figures 9 and 10 show two different ways the analog registers are accessed on the Transceiver. The Arbiter controls the register access traffic from an AHB master, an ARM Cortex processor, and SPI transactions from outside the chip. The addresses and offsets vary for both the masters. The address mapping and the timing information of the register bus are confidential and proprietary to ADI.

The digital testbench supports read-write transactions to analog sub-map through both ARM processor and SPI bus transactions. The connectivity verification is done through the SPI using an SPI transaction agent built into the testbench. The UVM testbench architecture employed in this project offers built-in exhaustive bit testing mechanism known as Bit Bash Testing.

3.2 Netlist

This methodology uses Cadence's Unified Netlister (UNL) supported by AMS simulator to netlist analog designs. The design hierarchy of the chip follows the same partition as in the example but at a much larger scale. The analog_top block contains many sub-blocks implemented in digital design flow of RTL design modeling and synthesis using standard cell library. This mixed nature of the analog_top block posed a tremendous challenge in obtaining a Verilog-AMS netlist. Analog_top is partitioned into four blocks with two of those containing the Transmit, Receive and observation channels' circuitry for multiple channels. The third block consists of RF converter circuits and a supporting Phase locked loop (PLL). The fourth block, named West, is a sophisticated clock generation and control circuitry. The West block encapsulates a PLL, RF clock generation circuit, proprietary phase detection circuits, biasing circuits and high-speed clock multiplexers.

As defined in section 2.2.3, creating a config view is the first step in preparing the design for netlisting and simulation. Given the massive size of the chip, the West block is chosen to be the block of interest to evaluate the methodology. The other blocks are chosen to be described as

higher-level behavioral SystemVerilog models. This decision helped reduce the complexity of the evaluation and the compile time.

3.3 Analog_top

The default AMS template was used for the config view of the analog_top with West block forced to use the schematic view, as the first attempt. The netlister tool complained about some of the lower level cells having multiple definitions. The blocks that were common in the West block and the other three blocks were conflicted with multiple definitions, and the netlister failed to create the netlist. After spending significant time on trying to netlist the analog_top as a single Verilog-AMS module, it was concluded that the tool does not have capabilities to handle the situation. The decision was made to netlist the analog_top with default AMS template and West block to use a Symbol view. Any block set to use a Symbol view the tool expects the definition of the module to be available during compilation. Figure 11 shows the instantiation of West block inside the analog_top module, but there is no definition for the module west in the netlist.

```
// AMS netlist generated by the AMS Unified netlister
// IC subversion:  IC6.1.7-64b.500.8
// IUS version: 15.20-s009
// Copyright(C) 2005-2009, Cadence Design Systems, Inc
// User: akanthar Pid: 80801
// Design library name:
// Design cell name: analog_top
// Design view name: config_epa4
// Solver: Spectre
...
module analog_top ( /*...
...
west i_west ( /*...

endmodule
```

Figure 11. Analog_top netlist with the instantiation of West block.

3.4 West block

The West block is netlisted with its own config view with all the cells forced to use schematic view except the digital sub-blocks and primitive cells. The digital cells are set to use

symbol view, and the definitions are made available during compilation as RTL models. The primitive cells are set to use the Spectre/SPICE models. These models are part of the Process Development Kit (PDK) provided by the Foundry. The analog_top does not require primitives since all the instantiations inside it are higher-level abstract models and may or may not be entirely functional ones.

3.5 Cadence runams netlist flow

The methodology employs Cadence's runams based netlist flow. Most of the system level simulation environments in the industry are SystemVerilog based UVM testbenches. The system level digital simulations for the transceiver chip is one such complex UVM testbench. Runams based netlisting offered more flexibility for integration of the generated netlist with the existing testbench. On the contrary, relying on ADE GUI based netlisting procedure lacks control and flexibility for integration of the netlist with a different design and testbench environment.

```
runams -cdslib ./cds.lib -lib <libname> -cell analog_top -view config_epa \
-netlist all -clean -rundir ./simulation_atop \
-simulate -amscontrol amscf.scs -path                              /spectre/3.5/ \
-modelfile toplevel.scs(top_tt) -modelfile toplevel.scs(adi_probe) -nograph
```

Figure 12. Runams netlister command-line options

The exact command to invoke the netlister is as shown in Figure 12. Runams command requires, at the least, the library, the cell and the view passed as options. If the library file cds.lib is not passed as an option, the tool assumes the first cds.lib file it finds as per the Cadence search hierarchy [7] and looks for the library, cell, and views in that file. The netlisting mode is specified using –netlist option. It can be set to incremental or all. The result directory for the netlister is specified using the –rundir option. The most important argument to the netlister is –modelfile. This

option tells the netlister where to find the models for the Spectre and SPICE models for the primitive cells and components in the circuit. The SPICE model file format allows sections to be specified within a library. It is important to specify the section in this option if one exists in the file. The library files from foundries have different sections in the simulation model libraries for different simulation corners. Without a section specified, the netlister fails to throw an error. Runams is rerun with –cell option having “west” as the argument and the –rundir option pointing to a different directory. The resulting netlist is the transistor level netlist of the West block.

The extract of netlist generated from the first runams command is shown in Figure 12. Despite not being a transistor level netlist, analog_top is a large entity due to the sheer magnitude of the transceiver chip. The topmost hierarchical module in the West block is called “west” as shown in Figure 13. The instantiation of this block is in the module analog_top as shown in Figure 11.

```
// AMS netlist generated by the AMS Unified netlister
// IC subversion:  IC6.1.7-64b.500.8
// IUS version: 17.04-s010
// Copyright(C) 2005-2009, Cadence Design Systems, Inc
// User: akanthar Pid: 40011
// Design library name:
// Design cell name: west
// Design view name: config_epa
// Solver: Spectre

`include "disciplines.vams"
`include "userDisciplines.vams"
...
module west (*...
...
endmodule
```

Figure 13. West block netlist extract

3.6 Endpoint Assertions

Endpoint assertion check is the heart of this methodology. These checks perform the actual connectivity between the origin endpoint and the destination endpoint of a control bit. SVA

concurrent assertions are used for the checks. SVA is chosen for developing assertions because it supports sequences involving multiple clock cycles, which is a required feature for monitoring bus transactions. The property construct of the SVA helped develop an automated way of generating the assertions. A macro is used to define the framework of an endpoint assertion. Complete assertion macro can be found in Appendix A. A simple script is used to generate the assertion for each register map uses this macro.

Analog Devices, Inc. uses a proprietary tool to maintain register information. All the characteristics of a register are collected into a single database using this tool. This centralization of information has enabled a more collaborative channel between different teams working on the same project. Digital design teams update the database with the register addresses, offsets, bit fields, enable fields for registers, access permission types and other details. Analog designers update the hierarchy of the control bits starting from the origin endpoint to the destination node endpoint. Verification engineers use this information to generate Register Abstraction Layer (RAL) model, which is a core component of modern testbenches. Applications and Firmware teams use the header files generated using the same database for register access in programming.

The SVA endpoint assertions have three major parts. The first part, like any other concurrent assertions, is a trigger. The trigger for endpoint assertions is the block's clock signal. The second part is a control switch, used to turn off assertions when required. The endpoint assertions are turned off during reset and when a particular block is powered down. The third and crucial part of the assertion is the check itself. The endpoint checks follow a format that as follows, "expression 1(if the expression is true) |-> ##(delay) expression 2(this expression must be true)". When the assertion is triggered, if the expression 1 is evaluated to be true, the assertion becomes active. The implication operator "|->" tells the tool to evaluate the expression 2 on the same trigger event. The ##(delay) operator is in place to delay the evaluation or expression 2 if required, which

is usually the case for a bus-driven register access. The expression 2 of endpoint checks monitors if the new voltage value at the node corresponds to the value written to the bit field.

3.7 Cadence $cadence_get_analog_value() or $cgav() functions

Cadence's Incisive and Xcelium simulators offer a system task to access values of analog nodes in Verilog-AMS netlist [5]. These system tasks are allowed to be used in any part of the simulation environment to access analog values. $cgav() accepts three input arguments. The first and compulsory argument is the hierarchy of the control bit as a string. The second argument is the index number if the signal probed is a multi-bit bus. By default, the probed signals are assumed as a single-bit node, unless an integer is passed for the index argument. The third, and last, argument is the quantity specifier. Quantity specifier is passed as a string and can be one of the following, "potential", "flow", "pwr", or "param". The $cgav function, if the hierarchy being probed exists, returns a real value that is used in assertions to check the current voltage value at the node. The voltage value of the measured node is with respect to the ground. Appendix A illustrates the implementation of endpoint assertion macro.

3.8 Assertion binding

The endpoint assertions are encapsulated inside a single module and bound to the design blocks using Verilog bind statement. Verilog LRM [6] defines bind construct can bind two design blocks making connections through common input and output ports. Bind construct is seldom used in the industry as glue logic to connect design blocks. More popular application of the bind statement is connecting the assertion modules to the design blocks. The bind statement must be in the top-level module and must be in the format shown in Figure 14 [6]. The connection between the assertion module and the block aux_pll is made with CLK and RST signals as common ports.

```
bind `aux_pll_REGMAP aux_pll_sva a_aux_pll_sva(`aux_pll_CLK,`aux_pll_RST);
```

Figure 14. Bind statement for analog block named aux_pll

3.9 Compile and Simulation

After completing the above steps, next step of the methodology is to integrate the pieces for compilation. The Xcelium simulator was invoked with a –f option with all the necessary command line options and arguments passed to the simulator in a text file. The entire arguments file is available in Appendix B. To make the connectivity verification exist alongside the existing simulation environment, the definitions of analog_top or the West block were not replaced or modified in any of the existing source files. Instead, the file with the definition of analog_top and the West block were directly passed to the simulator as command line arguments. Also, the command line arguments of digital simulation were retained as is. The two netlist files discussed in section 3.2.3 and an analog solver control file "amscf.scs" are added to the list of arguments.

3.10 Analog Control File amscf.scs

Cadence's Incisive and Xcelium (irun and xrun) simulators identify the nature of the design by inspecting all the design files passed as arguments. When the tool compiles the new netlist files, it identifies that the design is a mixed signal in nature and requires an Analog Solver for simulation. For compilation and simulation of Analog and Mixed-Signal designs, the simulator expects an analog control file passed as an argument that controls various characteristics of the analog solver. The control file follows Spectre netlist format. This file has the options to control parameters like simulator language, stop time, values to design variables, ignoring any known violations such as redefined parameters or duplicate subcircuit definitions, mixed-signal connectivity and design configurations for individual cells. Control file for the transceiver is as shown in Appendix C. The

most important part of the control file is the “amsd” block containing important settings for the methodology.

An “amsd” block contains statements that control AMS mechanisms during the elaboration phase of irun or xrun in the design verification flow. The simulation front-end (SFE) parser reads and applies elaboration statements during the elaboration phase and simulation statements during the simulation phase. Amsd blocks are valid only for AMS simulation [7]. The analog-digital connectivity requires the definition of interface element for analog to digital and vice versa signal conversion during simulation. This interface is defined in “amsd” block with “ie” construct. The ie statement specifies interface element parameters, optionally for a particular design unit [7]. If design unit is not specified, the elaborator applies the parameter settings to all interface elements globally. By default, the primitives are elaborated using SPICE or Spectre models and simulated. To change is behavior, “portmap” and “config” constructs are used. The portmap statement tells the AMS Designer simulator how a SPICE subcircuit interface should appear to the elaborator. The config statement specifies which definition to be used for particular cells or instances.

With multiple trials and experiments with the simulator arguments and “amscf.scs” configurations, the compilation of transceiver was successful and progressed to elaboration and simulation phases.

3.11 Bit Bash Test

The test required to test the connectivity of all the control bits required to read and write to an individual bit of the registers one by one. The read-write access is performed one at a time for the entire register map. The UVM methodology used in the digital design provides an inbuilt test sequence that performs this exhaustive read-write exercise. UVM tests are based on execution of sequences of transactions called uvm_sequence. The uvm_reg_bit_bash_seq fetches register

addresses and default values from RAL model, and exhaustively issues read-write transactions to each register bit. A UVM component named uvm_reg_adapter in the testbench converts the high-level read-write transactions to bus transactions [8]. The SPI bus agent in the digital testbench drives the bus signals to perform the actual read-write transactions to the registers. The results of the test are explained in Chapter 4.

Chapter 4

RESULTS AND OBSERVATIONS

4.1 Compilation and Elaboration

With West block netlisted at the transistor level, the compilation and elaboration took much longer than it did with digital simulations. The memory footprint that the elaboration process took was incredibly huge. Access to very powerful servers with over 200 gigabytes of memory and processor speed of 3.5 GHz were granted for the duration of this project. The simulator did not support multi-core execution of Mixed-Signal designs. Extracts of compilation and elaboration of both ring oscillator example and Transceiver is shown in the log files in Appendix D.

As a workaround to the lengthy elaboration and simulation time problem, assuming SPICE models take longer time during elaboration, AMS simulator's stub substitution feature was tried for the Transceiver. The amsd block in analog control file allows substitution of a Verilog-AMS stub in place of a SPICE model. As an updated workaround, the created stubs were saved and directly passed to the tool in consequent runs. There was a noticeable improvement in elaboration with this workaround.

4.2 Simulation results

The stub substitution, discussed in section 4.1, hardly had any effect on the simulation time. Log file extracts for both ring oscillator example and the Transceiver are shown in Appendix D. The simulation results of the ring oscillator are discussed in section 2.2.5. The test used for connectivity checks in Transceiver is called spi2_bit_bash_test, and it covers all the registers in the analog sub-map. The verification of read-write functionality is out of the scope of this project and covered as part of digital design verification.

Chapter 5

CONCLUSIONS AND FUTURE WORK

Though the methodology simplified the verification flow, it affected the performance significantly. The difficulties encountered with the tools during the creation of this methodology are to be resolved by Cadence Design Systems. The huge memory consumption and extremely slow simulation speed demands for closer inspection of the tool's AMS capabilities. It is reasonable to explore the possibilities of better performance, Than Spectre analog solver that AMS simulator inherently uses, from a different, SPICE based, simulator that supports Mixed-Signal simulations and provides an easy probe function like $cgav().

The compilation library management of the tool threw challenges during elaboration process that were managed with workarounds and extensive experimentation. Another solution would be to automate the generation of Verilog config block or the portmap and config constructs in the amsd block based on cell instances.

It is also possible to convert the Verilog-AMS netlist to a pure Verilog based hierarchy post-netlist. This flow would only require a digital simulator, significantly faster compared to the continuous-time analog solver.

Another vital aspect of simulation result is coverage. The digital verification offers a metric to quantify the verification efforts called functional coverage. Assertion coverage is a type of functional coverage that measures the percentage of assertions that were triggered and proven during the simulation time [6]. The Cadence tool that can used to analyze the coverage information is called Incisive Metrics Center (IMC).

APPENDIX A

ENDPOINT ASSERTION MACRO

```
//-----------------------------------------------
// asrt_en argument defination:
//-----------------------------------------------
// name : Bitfield name
// bf   : Bitfield in design hierarcy
// ep   : EndPoint connection of Bitfield
// rc   : Bitfield Reset condition
// ec   : Bitfield Enable Condition
// cc   : Bitfield Clock Condition
// td   : Bitfield path Time Delay
//-----------------------------------------------


//----------------------Generic Assertion Definition-----------------------------------
`define EP_HIER2STRING(x) `"x`"

`define asrt_en(name, bf, ep, rc, ec, cc, td)\
always@(cc) begin\
// ac   : bit field written with new value
ac = `RAL_submap.name.get() != `RAL_submap.name.peek();\
end\

property name;\
  real value_old, value_new;\
  @(posedge cc) disable iff (!ec || !rc)\
  (ac) |-> ##(td) ($cgav(ep) > 0.5;\
endproperty\
``name``_check : assert property(name);
```

APPENDIX B

XCELIUM XRUN –f ARFUMENT FILE

-clean
-64bit
./amscf.scs
-makelib stublib
cds_globals.vams
./device_stubs/vams/.vams*
-endlib
cds_globals.vams
netlist.vams

<paths to foundry model files>

+incdir+<path to analog model directories>

-y <path to analog model directories>

-reflib stublib
-reflib worklib
-sysv_ext +.v+.sv+.tv
-vlog_ext +.vg+.g+.bvrl+.vrl+.vp
+libext+.v+.sv+.tv+.vg+.g+.bvrl+.vrl+.vp
-ALLOWREDEFINITION
-timescale 1ns/1ps
-vtimescale 1ns/1ps
-uvm

-y <path to digital source files directories>

<digital systemverilog package file paths>

-define RTL \
-define NCV \
-licqueue \

+incdir+<digital verilog source code directories paths> \

****prorietary custom switches *********
-zlib 1 \
-notimingchecks \
-nospecify \
-add_seq_delay 1ps \

```
-nowarn SUBSCR
-gateloopwarn
syslvl_sim_top.sv
+UVM_TESTNAME=spi2_bit_bash_test
-mcdump
-simlogsize 100
-simincfile 2
+uvm_set_config_string="*,reg_slice_inst,${CUSTOM_uvm_slice_inst_OPTION}"
+uvm_set_config_string="*,reg_submap_inst,${CUSTOM_uvm_submap_inst_OPTION}"
```

APPENDIX C

ANALOG SOLVER CONTROL FILE AMSCF.SCS

```
simulator lang=spectre
amsd{
    ie vsup=1.0
    }
options options redefinedparams=ignore duplicatemodel=ignore duplicate_subckt=ignore
tran tran  stop=5m annotate=status
```

References

[1] Robert Cappel, Cathy Perry-Sullivan, “Yeild and cost challenges at 16nm and beyond”, February 2016. http://electroiq.com/blog/2016/02/yield-and-cost-challenges-at-16nm-and-beyond/

[2] Pranav Ashar, “Paradigm shift in Verification Methodology”, IEEE FMCAD, DOI: 10.1109/FMCAD.2016.7886652, Oct 2016.

[3] Mladen Nizic, “Mixed-Signal Methodology Guide”, ISBN – 978-1-300-03520-6

[4] Doulos, “SystemVerilog Assertions Tutorial, Concurrent assertions”, October 2016. www.doulos.com/knowhow/sysverilog/tutorial/assertions/

[5] Somasunder Kattepura Sreenath, “Verification of mixed-signal designs using SystemVerilog” assertions in co-simulation, Texas Insturments, SNUG 2010

[6] IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2012, December 2012

[7] Cadence design systems, <installation directory>/doc/amssimug/amssimug.pdf

[8] “Universal Verification Methodology (UVM) 1.2 User’s Guide”, Accellera Systems Initiative, October 2015